



FUNDAMENTAL INTRODUCTORY TO MACHINE LEARNING

1: PROBLEM FORMULATION

Clinical Objective: Predict the Presence or Absence of Atherosclerotic Heart Disease (AHD) to enable early detection and risk stratification

2: LOAD THE REQUIRED LIBRARIES AND DATA SET FOR EXPLORATORY DATA ANALYSIS

In [1]:

```
#!pip install xgboost
```

In [2]:

```
#### Data Importation and Visualization Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import time

#### Machine Learning Functionalities
from sklearn.model_selection import train_test_split, RandomizedSearchCV, StratifiedKFold
from sklearn.calibration import calibration_curve
from sklearn.preprocessing import OrdinalEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
```

```
#### Machine Learning Performance Functionalities
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
f1_score, roc_auc_score, confusion_matrix, precision_recall_curve, auc,
RocCurveDisplay, PrecisionRecallDisplay, ConfusionMatrixDisplay)

#### Functionality for Extreme Gradient Boosting
import xgboost as xgb
```

2.1. ENSURING REPRODUCIBILITY AND UNNECESSARY FILTERING WARNINGS

In [3]:

```
SEED = 42
np.random.seed(SEED)
warnings.filterwarnings('ignore')
```

2.2. LOADING THE REQUIRED DATASET

In [4]:

```
df = pd.read_csv("Heart.csv", na_values = ['NA', '?', ''])
df.head()
```

Out[4]:

	Age	Sex	ChestPain	RestBP	Chol	Fbs	RestECG	MaxHR	ExAng	Oldpeak	Slope	Ca	
0	63	1	typical	145	233	1	2	150	0	2.3	3	0.0	f
1	67	1	asymptomatic	160	286	0	2	108	1	1.5	2	3.0	noi
2	67	1	asymptomatic	120	229	0	2	129	1	2.6	2	2.0	revers
3	37	1	nonanginal	130	250	0	0	187	0	3.5	3	0.0	noi
4	41	0	nontypical	130	204	0	2	172	0	1.4	1	0.0	noi

3. EXPLORATORY DATA ANALYSIS

3.1. CHECK THE SHAPE AND DATA DISTRIBUTION

In [5]:

```
dir(df)
```

Out[5]:

```
['AHD',
 'Age',
 'Ca',
 'ChestPain',
 'Chol',
 'ExAng',
 'Fbs',
 'HD',
 'MaxHR',
 'Oldpeak',
 'RestBP',
 'RestECG',
 'Sex',
```

```
'Slope',
'T',
'Thal',
'_AXIS_LEN',
'_AXIS_ORDERS',
'_AXIS_TO_AXIS_NUMBER',
'_HANDLED_TYPES',
'__abs__',
'__add__',
'__and__',
'__annotations__',
'__array__',
'__array_priority__',
'__array_ufunc__',
'__arrow_c_stream__',
'__bool__',
'__class__',
'__contains__',
'__copy__',
'__dataframe__',
'__deepcopy__',
'__delattr__',
'__delitem__',
'__dict__',
'__dir__',
'__divmod__',
'__doc__',
'__eq__',
'__finalize__',
'__floordiv__',
'__format__',
'__ge__',
'__getattr__',
'__getattribute__',
'__getitem__',
'__getstate__',
'__gt__',
'__hash__',
'__iadd__',
'__iand__',
'__ifloordiv__',
'__imod__',
'__imul__',
'__init__',
'__init_subclass__',
'__invert__',
'__ior__',
'__ipow__',
'__isub__',
'__iter__',
'__itruediv__',
'__ixor__',
'__le__',
'__len__',
'__lt__',
'__matmul__',
'__mod__',
'__module__',
'__mul__',
```

```
'__ne__',
'__neg__',
'__new__',
'__or__',
'__pandas_priority__',
'__pos__',
'__pow__',
'__radd__',
'__rand__',
'__rdivmod__',
'__reduce__',
'__reduce_ex__',
'__repr__',
'__rfloordiv__',
'__rmatmul__',
'__rmod__',
'__rmul__',
'__ror__',
'__round__',
'__rpow__',
'__rsub__',
'__rtruediv__',
'__rxor__',
'__setattr__',
'__setitem__',
'__setstate__',
'__sizeof__',
'__str__',
'__sub__',
'__subclasshook__',
'__truediv__',
'__weakref__',
'__xor__',
'_accessors',
'_accum_func',
'_agg_examples_doc',
'_agg_see_also_doc',
'_align_for_op',
'_align_frame',
'_align_series',
'_append_internal',
'_arith_method',
'_arith_method_with_reindex',
'_attrs',
'_box_col_values',
'_can_fast_transpose',
'_check_copy_deprecation',
'_check_inplace_and_allows_duplicate_labels',
'_check_label_or_level_ambiguity',
'_clip_with_one_bound',
'_clip_with_scalar',
'_cmp_method',
'_combine_frame',
'_consolidate',
'_consolidate_inplace',
'_construct_axes_dict',
'_construct_result',
'_constructor',
'_constructor_from_mgr',
```

'_constructor_sliced',
'_constructor_sliced_from_mgr',
'_dir_additions',
'_dir_deletions',
'_dispatch_frame_op',
'_drop_axis',
'_drop_labels_or_levels',
'_ensure_valid_index',
'_find_valid_index',
'_flags',
'_flex_arith_method',
'_flex_cmp_method',
'_from_arrays',
'_from_mgr',
'_get_agg_axis',
'_get_axis',
'_get_axis_name',
'_get_axis_number',
'_get_axis_resolvers',
'_get_block_manager_axis',
'_get_bool_data',
'_get_cleaned_column_resolvers',
'_get_column_array',
'_get_index_resolvers',
'_get_item',
'_get_label_or_level_values',
'_get_numeric_data',
'_get_value',
'_get_values_for_csv',
'_getitem_bool_array',
'_getitem_multilevel',
'_getitem_slice',
'_gotitem',
'_hidden_attrs',
'_indexed_same',
'_info_axis',
'_info_axis_name',
'_info_axis_number',
'_info_repr',
'_init_mgr',
'_inplace_method',
'_internal_names',
'_internal_names_set',
'_is_homogeneous_type',
'_is_label_or_level_reference',
'_is_label_reference',
'_is_level_reference',
'_is_mixed_type',
'_iset_item',
'_iset_item_mgr',
'_iset_not_inplace',
'_iter_column_arrays',
'_ixs',
'_logical_func',
'_logical_method',
'_maybe_align_series_as_frame',
'_metadata',
'_mgr',
'_min_count_stat_function',

```
'_module_source',
'_needs_reindex_multi',
'_pad_or_backfill',
'_reduce',
'_reduce_axis1',
'_reindex_axes',
'_reindex_multi',
'_reindex_with_indexers',
'_rename',
'_replace_columnwise',
'_repr_data_resource_',
'_repr_fits_horizontal_',
'_repr_fits_vertical_',
'_repr_html_',
'_repr_latex_',
'_reset_cache',
'_sanitize_column',
'_select_dtypes_indices',
'_series',
'_set_axis',
'_set_axis_name',
'_set_axis_nocheck',
'_set_item',
'_set_item_frame_value',
'_set_item_mgr',
'_set_value',
'_setitem_array',
'_setitem_frame',
'_setitem_slice',
'_shift_with_freq',
'_should_reindex_frame_op',
'_slice',
'_stat_function',
'_stat_function_ddof',
'_to_dict_of_blocks',
'_to_latex_via_styler',
'_typ',
'_update_inplace',
'_validate_dtype',
'_values',
'_where',
'abs',
'add',
'add_prefix',
'add_suffix',
'agg',
'aggregate',
'align',
'all',
'any',
'apply',
'asfreq',
'asof',
'assign',
'astype',
'at',
'at_time',
'attrs',
'axes',
```

'between_time',
'bfill',
'boxplot',
'clip',
'columns',
'combine',
'combine_first',
'compare',
'convert_dtypes',
'copy',
'corr',
'corrwith',
'count',
'cov',
'cummax',
'cummin',
'cumprod',
'cumsum',
'describe',
'diff',
'div',
'divide',
'dot',
'drop',
'drop_duplicates',
'droplevel',
'dropna',
'dtypes',
'duplicated',
'empty',
'eq',
'equals',
'eval',
'ewm',
'expanding',
'explode',
'ffill',
'fillna',
'filter',
'first_valid_index',
'flags',
'floordiv',
'from_arrow',
'from_dict',
'from_records',
'ge',
'get',
'groupby',
'gt',
'head',
'hist',
'iat',
'idxmax',
'idxmin',
'iloc',
'index',
'infer_objects',
'info',
'insert',

'interpolate',
'isetitem',
'isin',
'isna',
'isnull',
'items',
'iterrows',
'itertuples',
'join',
'keys',
'kurt',
'kurtosis',
'last_valid_index',
'le',
'loc',
'lt',
'map',
'mask',
'max',
'mean',
'median',
'melt',
'memory_usage',
'merge',
'min',
'mod',
'mode',
'mul',
'multiply',
'ndim',
'ne',
'nlargest',
'notna',
'notnull',
'nsmallest',
'nunique',
'pct_change',
'pipe',
'pivot',
'pivot_table',
'plot',
'pop',
'pow',
'prod',
'product',
'quantile',
'query',
'radd',
'rank',
'rdiv',
'reindex',
'reindex_like',
'rename',
'rename_axis',
'reorder_levels',
'replace',
'resample',
'reset_index',
'rfloordiv',

'rmod',
'rmul',
'rolling',
'round',
'rpow',
'rsub',
'rtruediv',
'sample',
'select_dtypes',
'sem',
'set_axis',
'set_flags',
'set_index',
'shape',
'shift',
'size',
'skew',
'sort_index',
'sort_values',
'squeeze',
'stack',
'std',
'style',
'sub',
'subtract',
'sum',
'swaplevel',
'tail',
'take',
'to_clipboard',
'to_csv',
'to_dict',
'to_excel',
'to_feather',
'to_hdf',
'to_html',
'to_iceberg',
'to_json',
'to_latex',
'to_markdown',
'to_numpy',
'to_orc',
'to_parquet',
'to_period',
'to_pickle',
'to_records',
'to_sql',
'to_stata',
'to_string',
'to_timestamp',
'to_xarray',
'to_xml',
'transform',
'transpose',
'truediv',
'truncate',
'tz_convert',
'tz_localize',
'unstack',

```
'update',
'value_counts',
'values',
'var',
'where',
'xs']
```

In [6]:

```
print(f"Shape: {df.shape}")
```

Shape: (303, 15)

In [7]:

```
print(f"Column Names: {list(df.columns)}")
```

Column Names: ['Age', 'Sex', 'ChestPain', 'RestBP', 'Chol', 'Fbs', 'RestECG', 'MaxHR', 'ExAng', 'Oldpeak', 'Slope', 'Ca', 'Thal', 'AHD', 'HD']

3.2. CHECK THE NUMBER OF MISSING VALUES PER VARIABLE

In [8]:

```
df.isna().sum()
```

Out[8]:

Age	0
Sex	0
ChestPain	0
RestBP	0
Chol	0
Fbs	0
RestECG	0
MaxHR	0
ExAng	0
Oldpeak	0
Slope	0
Ca	4
Thal	2
AHD	0
HD	0

dtype: int64

3.3 CHECK THE DISTRIBUTION OF CASES FOR THE TARGET VARIABLE

In [9]:

```
df['AHD'].value_counts()
```

Out[9]:

AHD	
No	164
Yes	139

Name: count, dtype: int64

In [10]:

```
df['AHD'] = df['AHD'].map({'No': 0,
                           'Yes': 1})
print(df['AHD'].value_counts(normalize = True))
```

AHD	
0	0.541254

```
1    0.458746
Name: proportion, dtype: float64
```

```
In [11]:
```

```
print(df['HD'].value_counts(normalize = True))
```

```
HD
0    0.541254
1    0.458746
Name: proportion, dtype: float64
```

3.4. VISUALIZATION OF CASES FOR THE TARGET VARIABLE

```
In [12]:
```

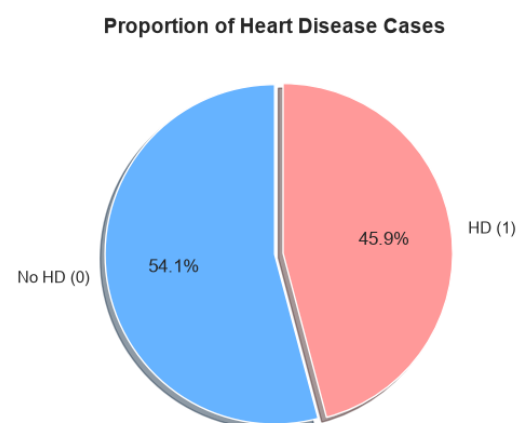
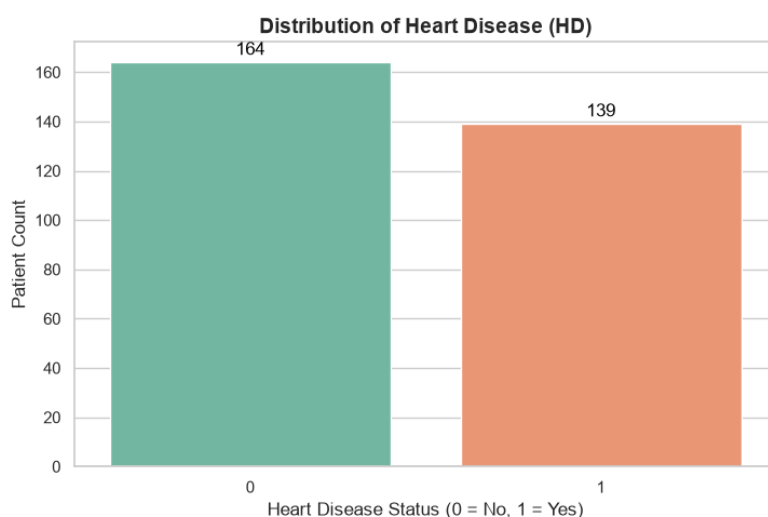
```
# Set global aesthetic
sns.set_theme(style="whitegrid", palette="muted")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# A. Countplot
sns.countplot(x='HD', data=df, ax=axes[0], palette='Set2')
axes[0].set_title('Distribution of Heart Disease (HD)', fontsize=14, fontweight='bold')
axes[0].set_xlabel('Heart Disease Status (0 = No, 1 = Yes)')
axes[0].set_ylabel('Patient Count')
# Add count labels on top of bars
for p in axes[0].patches:
    axes[0].annotate(f'{int(p.get_height())}', (p.get_x() + p.get_width() / 2., p.get_height() + 5),
                    ha='center', va='baseline', fontsize=12, color='black', xytext=(0,
                    textcoords='offset points'))

# B. Pie Chart
hd_counts = df['HD'].value_counts()
axes[1].pie(hd_counts, labels=['No HD (0)', 'HD (1)'], autopct='%1.1f%%',
            colors=['#66b3ff', '#ff9999'], startangle=90, explode=(0, 0.05), shadow=True)
axes[1].set_title('Proportion of Heart Disease Cases', fontsize=14, fontweight='bold')

plt.tight_layout()
plt.show()
```



3.5. EXTRACTION OF TARGET VARIABLE AND THE FEATURES

```
In [13]:
```

```
df.head()
```

Out[13]:

	Age	Sex	ChestPain	RestBP	Chol	Fbs	RestECG	MaxHR	ExAng	Oldpeak	Slope	Ca	Thal
0	63	1	typical	145	233	1	2	150	0	2.3	3	0.0	fast
1	67	1	asymptomatic	160	286	0	2	108	1	1.5	2	3.0	normal
2	67	1	asymptomatic	120	229	0	2	129	1	2.6	2	2.0	revers
3	37	1	nonanginal	130	250	0	0	187	0	3.5	3	0.0	normal
4	41	0	nontypical	130	204	0	2	172	0	1.4	1	0.0	normal

In [14]:

```
X = df.drop(columns = ['AHD', 'HD'])
X.head()
```

Out[14]:

	Age	Sex	ChestPain	RestBP	Chol	Fbs	RestECG	MaxHR	ExAng	Oldpeak	Slope	Ca	Thal
0	63	1	typical	145	233	1	2	150	0	2.3	3	0.0	fast
1	67	1	asymptomatic	160	286	0	2	108	1	1.5	2	3.0	normal
2	67	1	asymptomatic	120	229	0	2	129	1	2.6	2	2.0	revers
3	37	1	nonanginal	130	250	0	0	187	0	3.5	3	0.0	normal
4	41	0	nontypical	130	204	0	2	172	0	1.4	1	0.0	normal

In [15]:

```
y = df['AHD']
y.head()
```

Out[15]:

```
0    0
1    1
2    1
3    0
4    0
```

Name: AHD, dtype: int64

3.6.CATEGORIZE NUMERIC AND CATEGORICAL VARIABLES

In [16]:

```
num_features = ['Age', 'RestBP', 'Chol', 'MaxHR', 'Oldpeak', 'Ca']
cat_features = ['Sex', 'ChestPain', 'Fbs', 'RestECG', 'ExAng', 'Slope', 'Thal']
```

4. DATA PREPROCESSING

4.1. Create Numeric Tranformer

In [17]:

```
numeric_tranformer = Pipeline(steps = [
    ('imputer', SimpleImputer(strategy = 'median')),
```

```

    ('scaler', StandardScaler())
])

```

4.2. Categorical Transformer

In [18]:

```

categorical_transformer = Pipeline(steps = [
    ('imputer', SimpleImputer(strategy = 'most_frequent')),
    ('encoder', OrdinalEncoder(handle_unknown = 'use_encoded_value',
                               unknown_value = -1))
])

```

4.3. Create the Column Transformer

In [19]:

```

preprocessor = ColumnTransformer(
    transformers = [
        ('num', numeric_transformer, num_features),
        ('cat', categorical_transformer, cat_features)
    ],
    remainder = 'drop'
)

```

5. DATA PARTITIONING

In [20]:

```

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size = 0.2,
                                                    random_state = SEED,
                                                    stratify = y)

```

5.1 Check the Proportion of the Cases in the Training and Testing Set

In [21]:

```

train_dist = y_train.value_counts(normalize = True)
test_dist = y_test.value_counts(normalize = True)

proportion_df = pd.DataFrame({
    'Train (%)': train_dist,
    'Test (%)': test_dist
})*100

proportion_df.round(2)

```

Out[21]:

	Train (%)	Test (%)
AHD		
0	54.13	54.1
1	45.87	45.9

6. MODEL TRAINING, HYPERPARAMETER TUNING AND EVALUATION

6.1 CREATING THE PIPELINE FOR MACHINE LEARNING

In [22]:

```
pos_weight_train = (y_train == 0).sum()/(y_train == 1).sum()

### Logistic Regression
lr_pipeline = Pipeline(steps = [
    ('preprocessor', preprocessor),
    ('lr', LogisticRegression(random_state = SEED, max_iter = 1000,
                             class_weight = 'balanced'))
])

### K-Nearest Neighbors (KNN)
knn_pipeline = Pipeline(steps = [
    ('preprocessor', preprocessor),
    ('knn', KNeighborsClassifier())
])

### Support Vectors Machines
svm_pipeline = Pipeline(steps = [
    ('preprocessor', preprocessor),
    ('svm', SVC(random_state = SEED, probability = True,
                class_weight = 'balanced'))
])

### Decision Tree
dt_pipeline = Pipeline(steps = [
    ('preprocessor', preprocessor),
    ('dt', DecisionTreeClassifier(random_state = SEED,
                                  class_weight = 'balanced'))
])

### Random Forest
rf_pipeline = Pipeline(steps = [
    ('preprocessor', preprocessor),
    ('rf', RandomForestClassifier(random_state = SEED, n_jobs = -1))
])
```

6.2 SETTING UP THE PARAMETER GRID FOR THE ML MODELS

In [23]:

```
#### Logistic Regression Model (GLM)
param_dist_lr = {
    'lr__C': [0.01, 0.1, 1, 10, 100],
    'lr__penalty': ['l2'],
    'lr__solver': ['liblinear', 'saga']
}

#### K-Nearest Neighbors (KNN)
para_dist_knn = {
    'knn__n_neighbors': [3, 5, 7, 9, 11],
```

```

    'knn__weights': ['uniform', 'distance'],
    'knn__metric': ['euclidean', 'manhattan']
}

#### Support Vectors Machines (SVM)
param_dist_svm = {
    'svm__C': [0.1, 1, 10],
    'svm__kernel': ['linear', 'rbf'],
    'svm__gamma': ['scale', 'auto']
}

#### Decision Tree
param_dist_dt = {
    'dt__max_depth': [None, 5, 10, 15],
    'dt__min_samples_split': [2, 5, 10],
    'dt__min_samples_leaf': [1, 2, 4]
}

##### Random Forest
param_dist_rf = {
    'rf__n_estimators': [100, 200, 300, 400],
    'rf__max_depth': [None, 5, 10, 15],
    'rf__min_samples_split': [2, 5, 10],
    'rf__min_samples_leaf': [1, 2, 4],
    'rf__class_weight': ['balanced', 'balanced_subsamples']
}

```

6.3. MODEL TRAINING PROCESS

In [24]:

```

cv = StratifiedKFold(n_splits = 5, shuffle = True, random_state = SEED)

results = {}
training_times = {}

### Combine all the pipelines
all_pipelines = [
    ('Logistic Regression', lr_pipeline, param_dist_lr),
    ('KNN', knn_pipeline, para_dist_knn),
    ('SVM', svm_pipeline, param_dist_svm),
    ('Decision Tree', dt_pipeline, param_dist_dt),
    ('Random Forest', rf_pipeline, param_dist_rf)
]

for name, pipeline, params in all_pipelines:
    start = time.time()
    search = RandomizedSearchCV(
        estimator = pipeline,
        param_distributions = params,
        n_iter = 30,
        cv = cv,
        scoring = 'roc_auc',
        n_jobs = -1,
        random_state = SEED,
        refit = True
    )
    search.fit(X_train, y_train)
    training_times[name] = time.time() - start

```

```
results[name] = search
print(f"{name} Traiined Best CV ROC-AUC: {search.best_score_:.4f}| Time: {training_t
```

Logistic Regression Traiined Best CV ROC-AUC: 0.8996| Time: 15.48s
 KNN Traiined Best CV ROC-AUC: 0.8915| Time: 1.69s
 SVM Traiined Best CV ROC-AUC: 0.8982| Time: 1.32s
 Decision Tree Traiined Best CV ROC-AUC: 0.8183| Time: 2.22s
 Random Forest Traiined Best CV ROC-AUC: 0.8923| Time: 17.77s

7. PREDICTION AND MODEL EVALUATION

7.1. HOLD OUT CROSS VALIDATION

In [30]:

```
print("\n" + "="*60)
print("HOLDED-OUT TEST SET EVALUATION")
print("="*60)

eval_metrics = {}
for name, search in results.items():
    model = search.best_estimator_
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[: , 1]

    metrics = {
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred),
        'Recall': recall_score(y_test, y_pred),
        'F1-Score': f1_score(y_test, y_pred),
        'ROC-AUC': roc_auc_score(y_test, y_prob)
    }

    # Precision-Recall AUC (crucial for imbalanced/clinical tasks)
    prec, rec, _ = precision_recall_curve(y_test, y_prob)
    metrics['PR-AUC'] = auc(rec, prec)

    eval_metrics[name] = {**metrics, 'y_pred': y_pred, 'y_prob': y_prob, 'model': model}

print(f"\n📊 {name}")
for k, v in metrics.items():
    print(f"  {k:<12}: {v:.4f}")
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
=====
HOLDED-OUT TEST SET EVALUATION
=====
```

📊 Logistic Regression

Accuracy	: 0.8852
Precision	: 0.8387
Recall	: 0.9286
F1-Score	: 0.8814
ROC-AUC	: 0.9524
PR-AUC	: 0.9194

Confusion Matrix:

```
[[28  5]
 [ 2 26]]
```



KNN

Accuracy : 0.9016
Precision : 0.8929
Recall : 0.8929
F1-Score : 0.8929
ROC-AUC : 0.9686
PR-AUC : 0.9570

Confusion Matrix:

```
[[30  3]
 [ 3 25]]
```



SVM

Accuracy : 0.9016
Precision : 0.8667
Recall : 0.9286
F1-Score : 0.8966
ROC-AUC : 0.9545
PR-AUC : 0.9215

Confusion Matrix:

```
[[29  4]
 [ 2 26]]
```



Decision Tree

Accuracy : 0.7869
Precision : 0.7778
Recall : 0.7500
F1-Score : 0.7636
ROC-AUC : 0.8387
PR-AUC : 0.8374

Confusion Matrix:

```
[[27  6]
 [ 7 21]]
```



Random Forest

Accuracy : 0.9180
Precision : 0.8710
Recall : 0.9643
F1-Score : 0.9153
ROC-AUC : 0.9621
PR-AUC : 0.9519

Confusion Matrix:

```
[[29  4]
 [ 1 27]]
```

Prepare a Data Frame of Performance Metrics

In [31]:

```
results_df = pd.DataFrame({
    model_name: {
        'Accuracy': metrics['Accuracy'],
        'Precision': metrics['Precision'],
        'Recall': metrics['Recall'],
        'F1-Score': metrics['F1-Score'],
        'ROC-AUC': metrics['ROC-AUC'],
        'PR-AUC': metrics['PR-AUC']
    }
    for model_name, metrics in eval_metrics.items()
})
```

```
results_df = results_df.round(4)
results_df
```

Out[31]:

	Logistic Regression	KNN	SVM	Decision Tree	Random Forest
Accuracy	0.8852	0.9016	0.9016	0.7869	0.9180
Precision	0.8387	0.8929	0.8667	0.7778	0.8710
Recall	0.9286	0.8929	0.9286	0.7500	0.9643
F1-Score	0.8814	0.8929	0.8966	0.7636	0.9153
ROC-AUC	0.9524	0.9686	0.9545	0.8387	0.9621
PR-AUC	0.9194	0.9570	0.9215	0.8374	0.9519

Visualize the Confusion Matrix of the Best Model

In [47]:

```
# Confusion matrix: [[TN, FP], [FN, TP]]
cm = np.array([[29, 4],
               [1, 27]])

# Percentages of the total
total = cm.sum()
pct = (cm / total * 100).round(1)

# Create annotation text with label, value, and percentage
labels = np.array([
    [f"TN\n{cm[0,0]}\n({pct[0,0]:.1f}%)", f"FP\n{cm[0,1]}\n({pct[0,1]:.1f}%)"],
    [f"FN\n{cm[1,0]}\n({pct[1,0]:.1f}%)", f"TP\n{cm[1,1]}\n({pct[1,1]:.1f}%)"]
], dtype=object)

# Plot
plt.figure(figsize=(6, 4.5))
ax = sns.heatmap(
    cm,
    annot=labels,
    fmt='',
    cmap='Blues',
    cbar=False,
    linewidths=1,
    linecolor='black',
    annot_kws={'fontsize': 11, 'fontweight': 'bold'}
)

ax.set_title("Random Forest Confusion Matrix", fontsize=13, pad=10)
ax.set_xlabel("Predicted Label")
ax.set_ylabel("Actual Label")
ax.set_xticklabels(["Negative", "Positive"], rotation=0)
ax.set_yticklabels(["Negative", "Positive"], rotation=0)

plt.tight_layout()

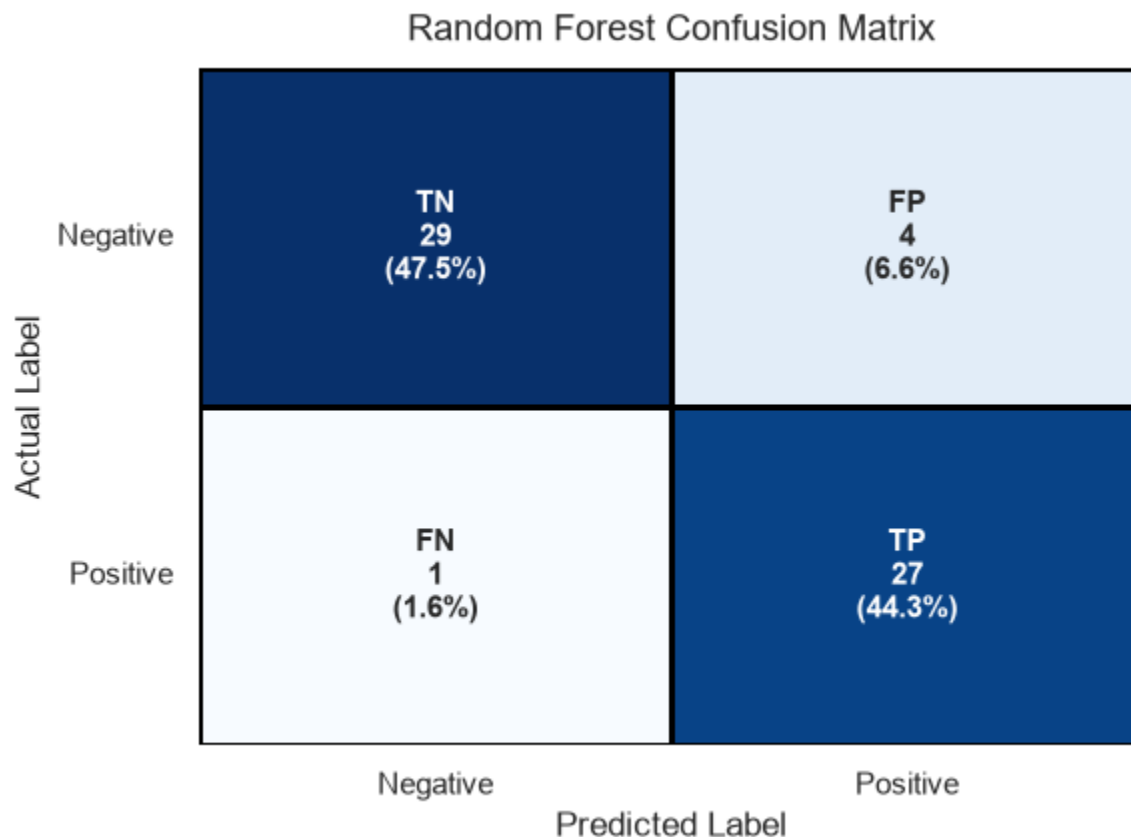
# Save the figure (must come before plt.show())
plt.savefig()
```

```

"8_confusion_matrix_random_forest.png",
dpi=300,
bbox_inches="tight"
)

plt.show()

```



KNN Model

In [33]:

```

# Confusion matrix: [[TN, FP], [FN, TP]]
cm = np.array([[30, 3],
               [3, 25]])

# Percentages of the total
total = cm.sum()
pct = (cm / total * 100).round(1)

# Create annotation text with label, value, and percentage
labels = np.array([
    [f"TN\n{cm[0,0]}\n({pct[0,0]:.1f}%)", f"FP\n{cm[0,1]}\n({pct[0,1]:.1f}%)"],
    [f"FN\n{cm[1,0]}\n({pct[1,0]:.1f}%)", f"TP\n{cm[1,1]}\n({pct[1,1]:.1f}%)"]
], dtype=object)

# Plot
plt.figure(figsize=(6, 4.5))
ax = sns.heatmap(
    cm,
    annot=labels,
    fmt='',
    cmap='Blues',
    cbar=False,

```

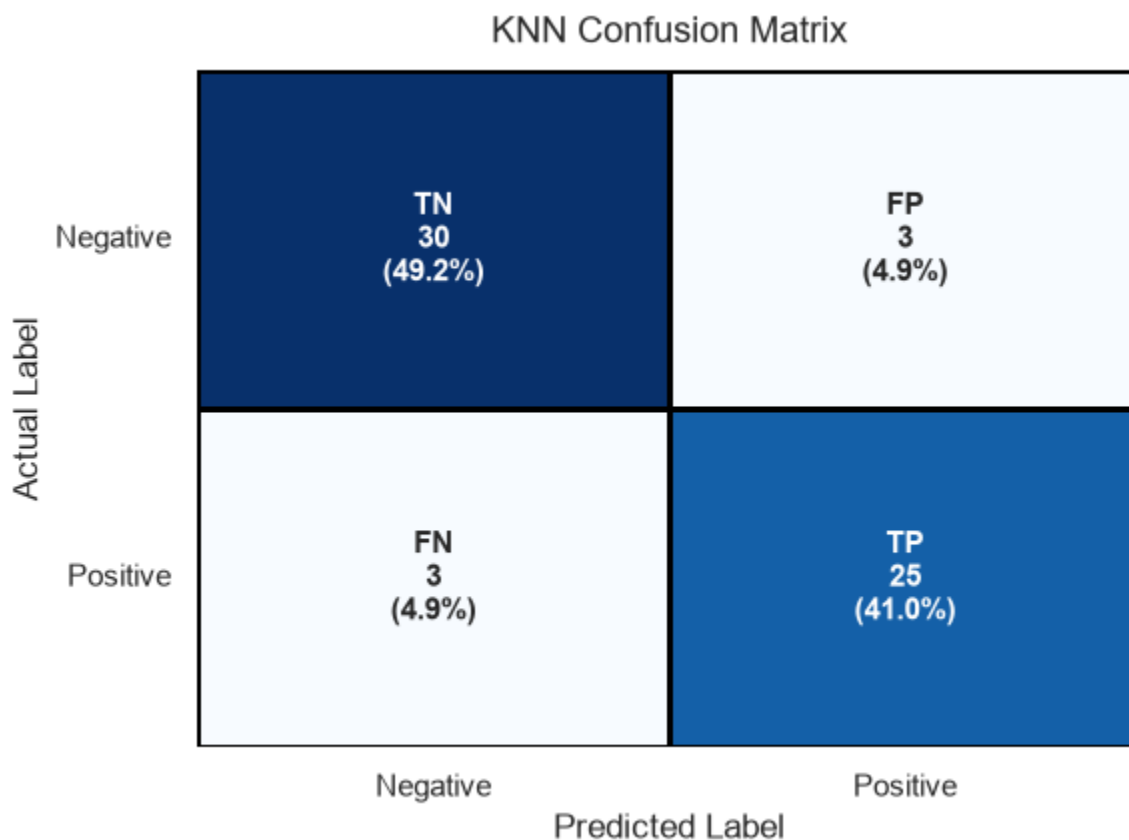
```

linewidths=1,
linecolor='black',
annot_kws={'fontsize': 11, 'fontweight': 'bold'}
)

ax.set_title("KNN Confusion Matrix", fontsize=13, pad=10)
ax.set_xlabel("Predicted Label")
ax.set_ylabel("Actual Label")
ax.set_xticklabels(["Negative", "Positive"], rotation=0)
ax.set_yticklabels(["Negative", "Positive"], rotation=0)

plt.tight_layout()
plt.show()

```



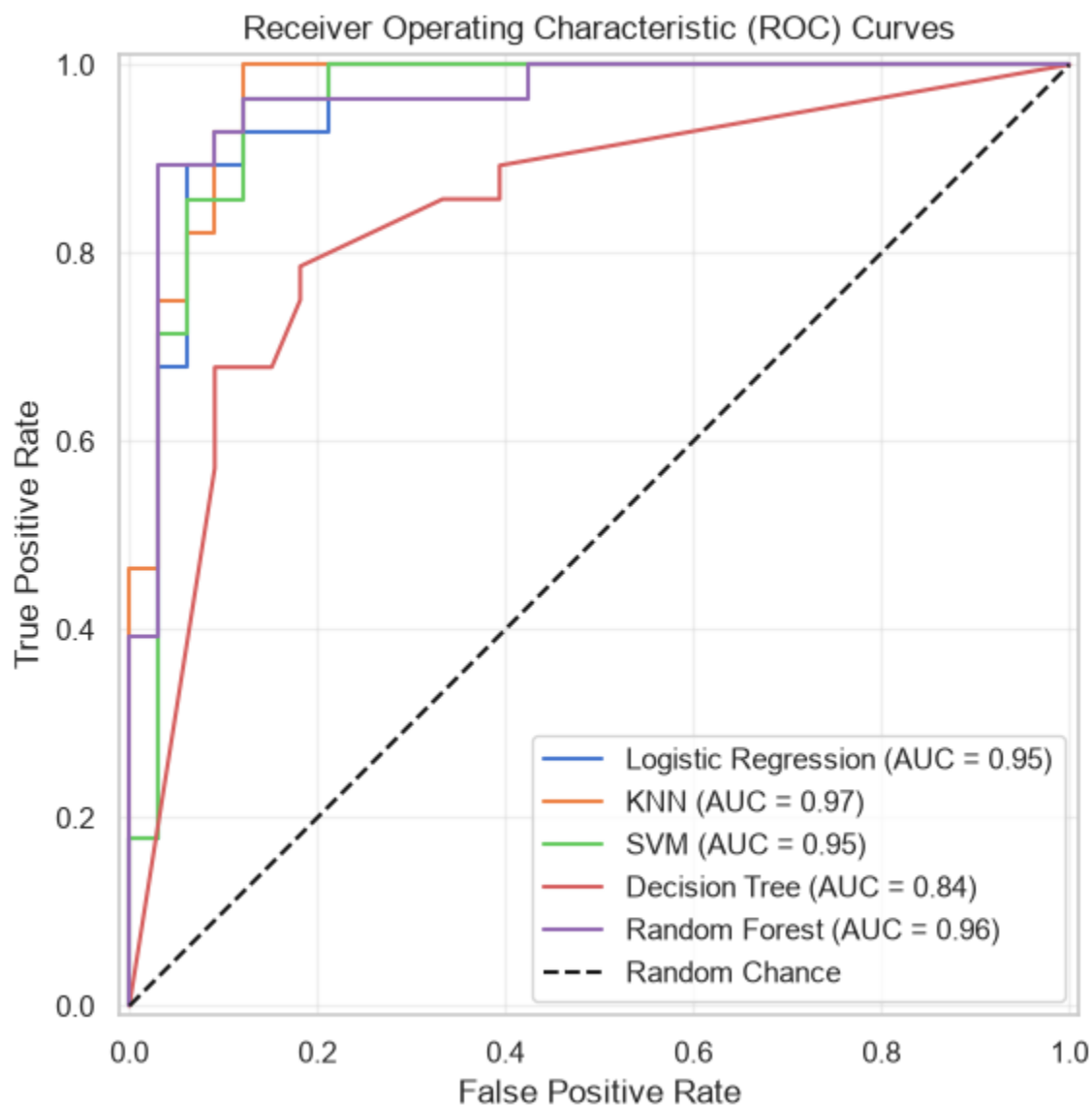
Received Operating Characteristic Curve and Area Under the Curve

In [36]:

```

plt.figure(figsize=(10, 6))
for name, res in eval_metrics.items():
    RocCurveDisplay.from_predictions(y_test, res['y_prob'], name=name, ax=plt.gca())
plt.plot([0, 1], [0, 1], 'k--', label='Random Chance')
plt.title('Receiver Operating Characteristic (ROC) Curves')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc='lower right')
plt.tight_layout()
plt.grid(alpha=0.3)
plt.savefig('2_roc_curves.png', dpi=300, bbox_inches='tight')
plt.show()

```



FEATURES IMPORTANCE

RANDOM FOREST MODEL

In [41]:

```
importances = eval_metrics['Random Forest']['model'].named_steps['rf'].feature_importances_

feature_names = num_features + cat_features
feat_imp = pd.Series(importances, index=feature_names).sort_values(ascending=True)
feat_imp
```

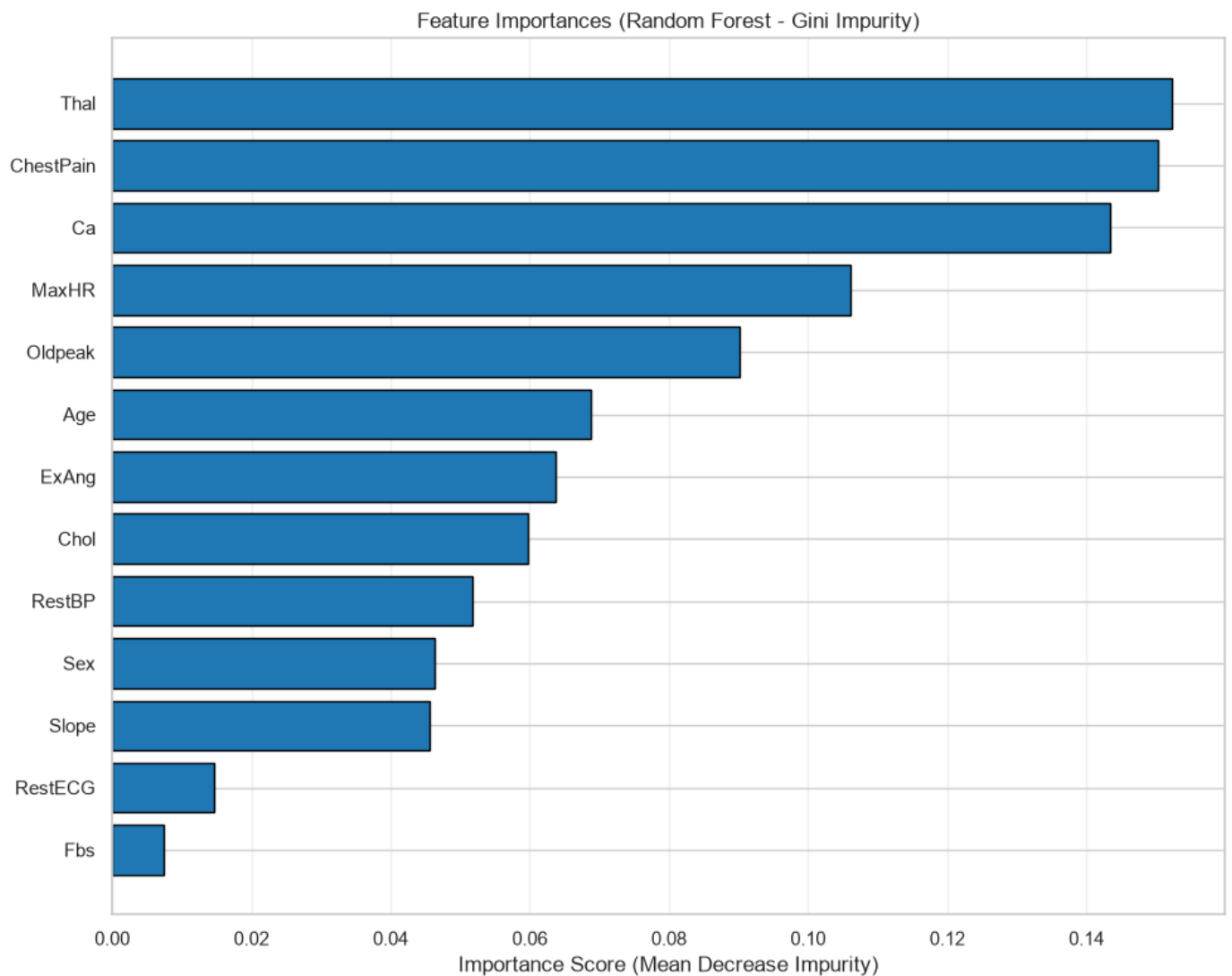
Out[41]:

Fbs	0.007464
RestECG	0.014574
Slope	0.045617
Sex	0.046359
RestBP	0.051777
Chol	0.059683
ExAng	0.063704
Age	0.068795
Oldpeak	0.090211
MaxHR	0.106002
Ca	0.143348

```
ChestPain    0.150196
Thal         0.152267
dtype: float64
```

In [42]:

```
plt.figure(figsize=(10, 8))
plt.barh(feat_imp.index, feat_imp.values, color='#1f77b4', edgecolor='black')
plt.title('Feature Importances (Random Forest - Gini Impurity)')
plt.xlabel('Importance Score (Mean Decrease Impurity)')
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.savefig('7_feature_importance_rf.png', dpi=300, bbox_inches='tight')
plt.show()
```



KNN-MODEL

In [44]:

```
dir(KNeighborsClassifier)
```

Out[44]:

```
['__abstractmethods__',  
'__annotations__',  
'__class__',  
'__delattr__',  
'__dict__',
```

```

'__dir__',
'__doc__',
'__eq__',
'__format__',
'__ge__',
'__getattr__',
'__getstate__',
'__gt__',
'__hash__',
'__init__',
'__init_subclass__',
'__le__',
'__lt__',
'__module__',
'__ne__',
'__new__',
'__reduce__',
'__reduce_ex__',
'__repr__',
'__setattr__',
'__setstate__',
'__sizeof__',
'__sklearn_clone__',
'__sklearn_tags__',
'__slotnames__',
'__str__',
'__subclasshook__',
'__weakref__',
'_abc_impl',
'_check_algorithm_metric',
'_doc_link_module',
'_doc_link_template',
'_doc_link_url_param_generator',
'_fit',
'_get_class_level_metadata_request_values',
'_get_doc_link',
'_get_fitted_attr_html',
'_get_metadata_request',
'_get_param_names',
'_get_params_html',
'_html_repr',
'_kneighbors_reduce_func',
'_parameter_constraints',
'_repr_html_',
'_repr_html_inner',
'_repr_mimebundle_',
'_validate_params',
'fit',
'get_metadata_routing',
'get_params',
'kneighbors',
'kneighbors_graph',
'predict',
'predict_proba',
'score',
'set_params',
'set_score_request']

```

In [45]:

```

from sklearn.inspection import permutation_importance
import pandas as pd

# Extract the trained pipeline
knn_model = eval_metrics['KNN']['model']

# Calculate permutation importance
perm_importance = permutation_importance(
    knn_model,
    X_test,
    y_test,
    n_repeats=20,
    random_state=42,
    scoring='accuracy'
)

# Feature names
feature_names = num_features + cat_features

# Create Series
feat_imp = pd.Series(
    perm_importance.importances_mean,
    index=feature_names
).sort_values(ascending=True)

print(feat_imp)

```

```

Sex            0.004098
MaxHR          0.004918
Oldpeak        0.010656
RestBP         0.011475
Fbs            0.012295
Age            0.014754
Ca             0.018033
ChestPain      0.031148
ExAng          0.035246
RestECG        0.039344
Thal           0.039344
Chol           0.044262
Slope          0.079508
dtype: float64

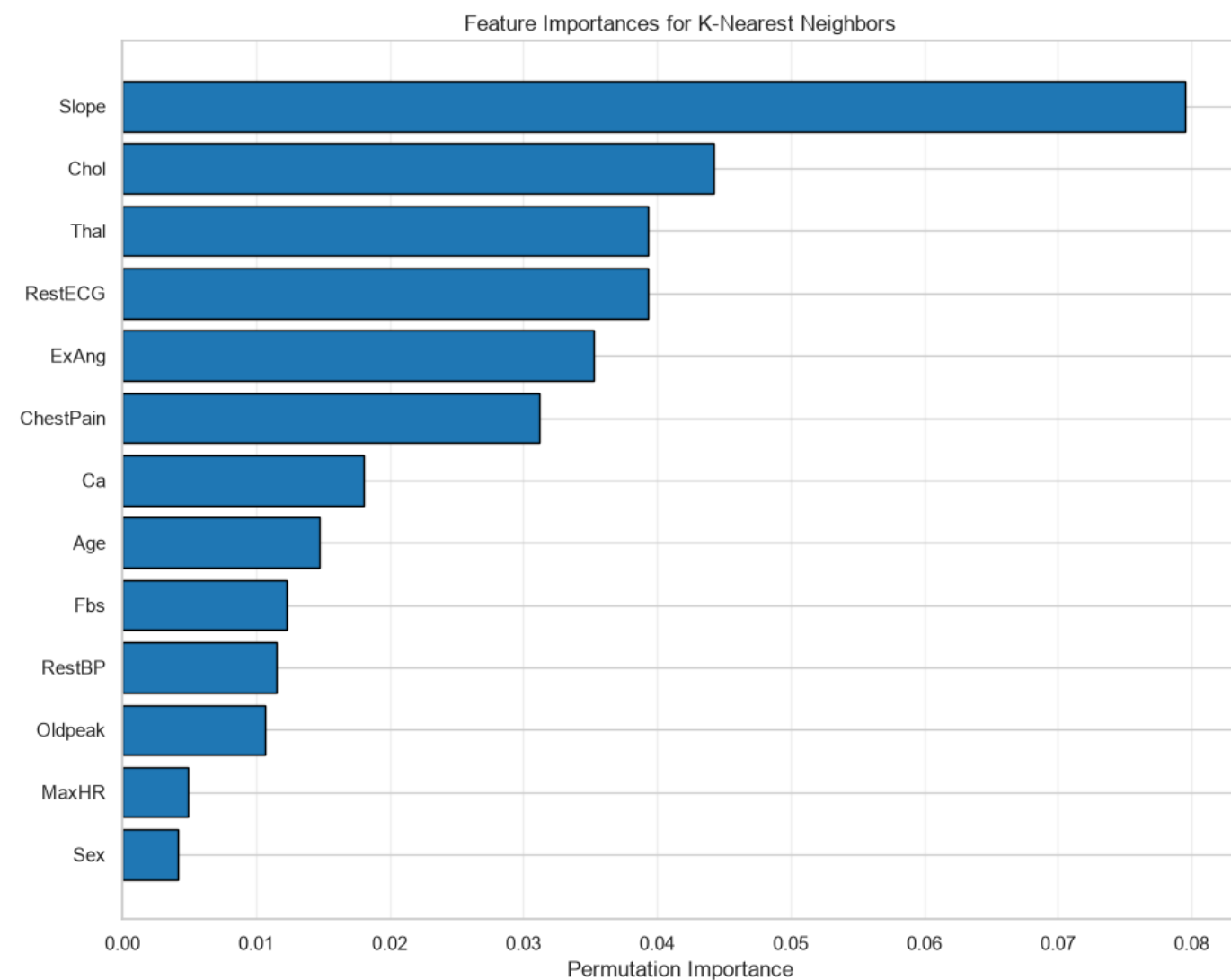
```

In [46]:

```

plt.figure(figsize=(10, 8))
plt.barh(feat_imp.index, feat_imp.values, color='#1f77b4', edgecolor='black')
plt.title('Feature Importances for K-Nearest Neighbors')
plt.xlabel('Permutation Importance')
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.savefig('7_feature_importance_knn.png', dpi=300, bbox_inches='tight')
plt.show()

```



In []: